

PlatePlanner

Models, Ingredient Breakdown & Recipe Generation

A deep-dive into the ML pipeline — from raw ingredient text to ranked recipe results

San Jose State University · CMPE 295B · April 2026

1. System Overview

PlatePlanner's recommendation engine is not a single model — it is a **multi-stage pipeline** where each component handles the part of the problem it is best suited for. SpaCy parses raw text at ingest time, a sentence-transformer handles semantic similarity at query time, a discrete overlap function handles exact ingredient matching, Neo4j enforces dietary safety, and an LLM fires only as a last resort to synthesise novel recipes when the corpus falls short.

2.2M+ Recipes indexed	384 Embedding Dims	<20ms FAISS Latency	+23% Recall Boost	3 LLM Providers
--------------------------	-----------------------	------------------------	----------------------	--------------------

Key pipeline statistics

2. Ingredient Breakdown

Before any ML model sees an ingredient, it passes through two preprocessing stages: **normalization** (at query time) and **Named Entity Recognition** (at ingest time).

2.1 Text Normalization (ingredient_normalizer.py)

Every ingredient string — whether from a user query or stored recipe — is cleaned by the same normalization pipeline:

Stage	Value
Input	"1 cup diced tomatoes"
Lowercase + strip non-alpha	"1 cup diced tomatoes"
WordNinja split	["1", "cup", "diced", "tomatoes"]
Remove units	cup, tbsp, oz, ml, lb, tsp ... → dropped
Remove descriptors	diced, chopped, minced, sliced ... → dropped
Remove stopwords / blacklist	of, fresh, large, small ... → dropped
Result	"tomato"

Table 1 — Normalization pipeline for '1 cup diced tomatoes'

2.2 Named Entity Recognition (SpaCy en_core_web_sm)

SpaCy NER runs **once during data ingestion** across all 2.2M recipes. The parsed ingredient tokens are stored in the ner column of the SQLite database so query-time lookup requires only a fast literal_eval() rather than live NLP inference.

- **87% precision** on 1.5M unstructured ingredient strings
- **82% recall** on the same evaluation set
- Handles fractions, unicode vulgar fractions (½ ¾ ■), ranges (1–2 cups)
- Strips brand names, preparation notes, temperature hints

2.3 Fuzzy Ingredient Matching (ingredient_matcher.py)

For shopping list consolidation and pantry matching, a three-level similarity check resolves ingredient name variants:

Check	Logic
1. Exact match	After normalization — fastest path
2. Synonym lookup	Hardcoded dict: tomato ↔ tomatoes ↔ cherry tomato ↔ roma tomato
3. Fuzzy ratio	Max of fuzz_ratio, partial_ratio, token_sort_ratio ≥ 85%

Table 2 — Three-level ingredient similarity check

3. Semantic Embedding Model

The semantic similarity backbone is **all-MiniLM-L6-v2** from the sentence-transformers library. At query time the user's ingredient list is joined into a single string and encoded into a 384-dimensional dense vector.

Property	Value
Model	all-MiniLM-L6-v2 (base) / finetuned-recipe-encoder (if present)
Output dimension	384-d float32 dense vector
Normalization	L2-normalized before FAISS search (enables cosine similarity via dot product)
Hardware	CPU inference, ~30–50ms per query
Auto-selection	Checks for finetuned path first; falls back to base model

Table 3 — Sentence-transformer model properties

3.1 Query Encoding (step-by-step)

```
query = " ".join(ingredients)      # ["butter","sugar","flour"] → "butter sugar flour"
query_vec = model.encode([query])   # shape: (1, 384)
faiss.normalize_L2(query_vec)       # L2 norm → cosine similarity via inner product
```

3.2 Fine-tuned Recipe Encoder (optional)

If a fine-tuned domain model is present at `src/data/models/recipe_suggestion/finetuned-recipe-encoder/`, it is loaded automatically. This model is adapted on food/recipe text and produces embeddings that better capture culinary concepts (e.g., 'braise' and 'slow-cook' map closer together than in the general model). A matching fine-tuned FAISS index (`recipe_index_finnetuned_opt.faiss`) is also used when available.

4. FAISS Vector Index

All 2.2M recipe embeddings are stored in a **FAISS IVF-PQ** (Inverted File + Product Quantization) index. This is the key engineering decision that makes the system practical: it reduces memory from ~12 GB (flat exact index) to ~90 MB while keeping query latency under 20ms.

Property	Value
Index type	IVF-PQ (Inverted File Index + Product Quantization)
File	recipe_index_opt.faiss
Size on disk	~90 MB (vs ~12 GB for flat exact index)
Vectors indexed	2,231,150 recipe embeddings
Query (k=500)	<20ms p95 on CPU
Quantization	8-bit per component; reduces RAM ×60 with minimal recall loss
Fallback	recipe_index.faiss (flat, exact) used for offline evaluation only

Table 4 — FAISS index properties

4.1 Search Execution

```
distances, indices = index.search(query_vec, k=500) # top-500 ANN
found_ids = [int(i) for i in indices[0] if i >= 0] # filter invalid

# Batch fetch from SQLite
placeholders = ",".join("? " * len(found_ids))
cursor.execute(
    f"SELECT id, title, ingredients, directions, ner FROM recipes
    WHERE id IN ({placeholders})",
    found_ids
)
```

The wide net of 500 candidates is intentional — downstream filtering (dietary, overlap, deduplication) will reduce this to the final top-N, so casting broadly ensures good coverage.

5. Hybrid Ranking Algorithm

Raw cosine similarity alone misses recipes that share many of the user's exact ingredients but differ in phrasing or cooking style. A discrete ingredient-overlap score is blended in to reward exact matches:

$$\text{combined_score} = 0.4 \times \text{cosine_similarity} + 0.6 \times \text{ingredient_overlap}$$

5.1 Cosine Similarity (semantic component, 40%)

The FAISS distance returned for each candidate is already a normalized inner product (since both query and index vectors are L2-normalized), equivalent to cosine similarity. It captures *meaning* — 'braise short ribs' and 'slow-cook beef' score close even if they share no words.

5.2 Ingredient Overlap Score (discrete component, 60%)

```
# Normalize user inputs
norm_inputs = {x.lower().strip() for x in ingredients}
```

```
# For each candidate recipe, count matching ingredients
overlap_set = set()
for r_ing in recipe_ingredient_list:
    for u_ing in norm_inputs:
        if u_ing in r_ing.lower():          # substring match
            overlap_set.add(r_ing)
            break

overlap_score = len(overlap_set) / max(len(ingredients), 1)
```

5.3 Minimum Overlap Filter

Recipes with fewer than **2 matching ingredients** are dropped entirely before scoring. This hard cutoff eliminates semantically adjacent but ingredient-incompatible results (e.g., 'chocolate cake' appearing for a 'butter lettuce' query).

Method	Notes	Recall vs Baseline
Semantic only	Baseline	0%
Overlap only	Misses semantic variants	—
Hybrid (0.4/0.6)	Production	+23% recall

Table 5 — Ranking method comparison

6. Dietary Classification & Allergen Detection

After ranking, every candidate recipe passes through a rule-based `DietaryClassifier`. No external model is needed — it uses **token matching with compound-ingredient exceptions** to avoid false positives like flagging 'coconut milk' as dairy.

6.1 Classification Logic

Tag	Blocked tokens	Exception example
is_vegan	No meat, poultry, seafood, dairy, eggs, honey	"coconut milk" is vegan
is_vegetarian	No meat, poultry, seafood	"fish sauce" fails; "soy sauce" passes
is_gluten_free	No wheat, barley, rye, malt	"soy sauce" fails (contains wheat)
is_dairy_free	No milk, butter, cream, cheese, yogurt	"coconut cream" passes

Table 6 — Dietary tags and exception handling

6.2 Allergen Detection

Allergen	Trigger tokens (sample)	Exceptions
peanut	peanut, peanut butter, peanut oil	—
tree_nut	almond, walnut, cashew, pecan, pistachio...	coconut (not a tree nut)
dairy	milk, butter, cream, cheese, yogurt...	coconut milk, oat milk, almond milk
gluten	wheat, flour, bread, pasta, barley...	rice flour, cornstarch
soy	soy, tofu, edamame, miso, tempeh...	—
shellfish	shrimp, crab, lobster, oyster...	—
fish	salmon, tuna, cod, tilapia, anchovy...	—
egg	egg, eggs, mayonnaise...	egg-free mayo

Table 7 — Allergen detection tokens and exceptions

6.3 Word-Boundary Safety (Neo4j)

In the ontology filtering stage, Neo4j Cypher uses word-boundary regex to prevent partial token matches:

```
-- "nut" must not match "nutmeg" or "coconut"
WHERE ing =~ ('(?i).*\\b' + term + '\\b.*')

-- "egg" must not match "eggplant"
WHERE ing =~ ('(?i).*\\begg\\b.*')
```

7. Three-Tier Federated Recipe Search

The `POST /suggest_recipes` endpoint orchestrates three increasingly powerful (and expensive) retrieval sources. Each tier is only invoked when the previous one falls short.

Tier	Source	Trigger condition	Latency	Coverage
Tier 1	Local FAISS	Always	~20ms	2.2M recipes, fully offline
Tier 2	Spoonacular API	score < 0.35 OR cuisine requested OR not enough results	~100ms	Millions of web recipes
Tier 3	LLM Generation	fewer than 2 results OR score < 0.25	~2–5s	Synthesises novel recipes from scratch

Table 8 — Three-tier retrieval system

7.1 Deduplication Between Tiers

Before merging local and Spoonacular results, a fuzzy title normalizer removes duplicates. Titles are lowercased, punctuation stripped, common words removed, and then compared with an 85% similarity threshold.

```
def _normalize_title(title: str) -> str:
    t = title.lower()
    t = re.sub(r'^a-z0-9\s', '', t)      # strip punctuation
    t = re.sub(r'\b(the|a|an|with|and|or)\b', '', t) # stopwords
    return t.strip()

def _is_duplicate(title, seen_titles, threshold=85):
    norm = _normalize_title(title)
    return any(fuzz.ratio(norm, s) >= threshold for s in seen_titles)
```

8. LLM-Powered Recipe Generation

When tiers 1 and 2 cannot return sufficient high-confidence results, the **RAGService** calls a large language model to synthesise a recipe from scratch. Three backends are supported with automatic provider detection:

Provider	Activation	Model	Trade-off
OpenAI	OPENAI_API_KEY set	gpt-4o-mini (default) or gpt-4	Highest quality
Google Gemini	GOOGLE_API_KEY set	gemini-2.0-flash (with fallback chain)	Fast, free tier available
Ollama (local)	Default fallback	llama3 or configurable	Private, no cost, slower

Table 9 — LLM backend options

8.1 Generation Prompt

The prompt is dynamically assembled from the user's ingredients and dietary flags. A strict JSON-only instruction prevents markdown wrapping:

```
prompt = (
    f"Generate {top_n} complete recipe(s) using: {ing_str}.\n"
    f"Cuisine: {cuisine_str}. Dietary requirements: {dietary_str}.\n\n"
    f"Return ONLY a valid JSON array. Each element must have:\n"
    f'  \"title\": string\n'
    f'  \"ingredients\": array (only user's ingredients that are used)\n'
    f'  \"all_ingredients\": array (full list with quantities)\n'
    f'  \"directions\": string (numbered steps)\n'
    f'  \"cuisine\": array of strings\n\n'
    f"No markdown, no explanation – just the JSON array."
)

system = (
    "You are a professional chef and recipe writer. "
    "Always respond with valid, parseable JSON only."
)
```

8.2 Response Parsing & Fallback

```
raw = await self.llm.generate(prompt, system, temperature=0.8)

# Strip markdown code fences if LLM included them
if raw.startswith("```"):
    raw = raw.split("```")[1]
    if raw.startswith("json"):
        raw = raw[4:]

parsed = json.loads(raw.strip())
if not isinstance(parsed, list):
    parsed = [parsed]
```

8.3 LLM Result Scoring

LLM-generated recipes are injected with fixed scores so they rank below high-confidence local results but above weak local matches:

```
{
  "semantic_score": 0.5,    # neutral
  "overlap_score": 1.0,    # all requested ingredients were used
}
```



```
"combined_score": 0.7,    # above LLM_TRIGGER_THRESHOLD (0.25)
"source":                "llm"
}
```

9. Recipe Adaptation via RAG

Beyond generation, the RAGService also **adapts existing recipes** to meet dietary needs or work around missing ingredients. This is the 'Retrieval-Augmented Generation' component: the retrieved recipe is the context, and the LLM's job is to reason about changes.

9.1 Adaptation Prompt Assembly

```
context_parts = [
    f"Recipe: {recipe_title}",
    f"Ingredients: {'', '.join(recipe_ingredients))",
    f"Directions: {recipe_directions[:500]}",
]
if pantry:
    context_parts.append(f"User has: {'', '.join(pantry))")
if missing_ingredients:
    context_parts.append(f"Missing: {'', '.join(missing_ingredients))")
if substitutions:
    subs = "; ".join(f"{k} -> {v}" for k,v in substitutions.items())
    context_parts.append(f"Suggested substitutions: {subs}")
```

9.2 Prompt Branching by Intent

Mode	Trigger	Prompt instruction
Dietary adaptation	"dietary" param set	"Adapt this recipe to be vegan. Explain each substitution."
Missing ingredient workarounds	"missing_ingredients" set	"Suggest how to adapt using what they have."
General tips	No specific params	"Provide cooking tips and suggestions."

Table 10 — Recipe adaptation prompt branching

10. Hybrid Recommendation Pipeline (Meal Plans)

For full meal plan generation, a deeper 4-stage pipeline runs beyond the basic `suggest_recipes` endpoint. It layers ontology safety, nutritional fitness, and personalized explanation on top of the FAISS retrieval.

Stage	Name	What happens
Stage 1	Semantic Retrieval	FAISS returns top-100 candidates for the query
Stage 2	Ontology Filtering	Neo4j Cypher removes recipes containing restricted ingredients (word-boundary safe)
Stage 3	Nutrient Scoring	$\text{Fitness score} = \text{semantic_score} - (\text{calorie_distance} / 500) \times 0.5$
Stage 4	Explanation Generation	LLM writes a 2-3 sentence personalised explanation of why the top recipe fits the user

Table 11 — Hybrid recommendation pipeline stages

10.1 Nutritional Fitness Score

```
def rank_by_nutrition(safe_recipes, user_goals):
    target_cals = user_goals.get("target_calories", 600)

    for recipe in safe_recipes:
        recipe_cals = recipe.get("calories_per_serving") or 600.0
        cal_distance = abs(recipe_cals - target_cals)
        cal_penalty = min(cal_distance / 500.0, 1.0) # normalize
        sem_score = recipe.get("semantic_score", 0.0)

        recipe["fitness_score"] = sem_score - (cal_penalty * 0.5)

    return sorted(safe_recipes, key=lambda x: x["fitness_score"], reverse=True)
```

10.2 Explanation Prompt (RAG)

```
prompt = (
    f"Selected Recipe: {recipe['title']}\n"
    f"Fitness Score: {recipe['fitness_score']:.2f}\n"
    f"User Goals: {goals}\n"
    f"Restrictions avoided: {restrictions}\n\n"
    "Write a friendly 2-3 sentence explanation to the user about why "
    "this recipe was selected. Highlight how it respects their dietary "
    "restrictions and aligns with their fitness/macro goals."
)
```

11. End-to-End Data Flow

#	Stage	Detail
1	User sends POST <code>/suggest_recipes</code>	<code>{"ingredients": ["butter", "sugar", "flour"], "is_vegan": false}</code>
2	Query embedding	"butter sugar flour" → SentenceTransformer → 384-d vector → L2 normalize
3	FAISS ANN search	<code>index.search(query_vec, k=500)</code> → top-500 recipe IDs + distances (<20ms)
4	SQLite batch fetch	<code>SELECT id, title, ingredients, directions, ner FROM recipes WHERE id IN (...)</code>
5	Ingredient parse	<code>literal_eval(ner_field)</code> → list of pre-parsed ingredient tokens

#	Stage	Detail
6	Overlap scoring	Substring match of user inputs vs recipe tokens → overlap_score
7	Hybrid score	$0.4 \times \text{cosine_sim} + 0.6 \times \text{overlap} \rightarrow \text{combined_score}$; drop if < 2 matches
8	Dietary classification	DietaryClassifier.classify(ingredients) → tags + allergens
9	Hard filter	Drop recipes violating requested dietary restrictions
10	Tier 2 decision	If max_score < 0.35 or results < top_n → call Spoonacular API
11	Deduplication	Fuzzy title matching; merge local + external; sort by combined_score
12	Tier 3 decision	If results < 2 or max_score < 0.25 → call LLM to generate recipes
13	Final sort & rank	Sort by combined_score desc; assign ranks 1..N
14	Return	List[RecipeResult] with scores, tags, source, ingredients, directions

Table 12 — Complete end-to-end data flow

12. Configuration Reference

Parameter	Default	Effect
BASE_MODEL_NAME	all-MiniLM-L6-v2	Sentence-transformer base model
FINETUNED_MODEL_PATH	src/data/models/.../finetuned-recipe-encoder	Domain-tuned model (optional)
OPT_INDEX_PATH	recipe_index_opt.faiss	IVF-PQ quantised FAISS index
raw_k	500	ANN search width before filtering
min_overlap	2	Minimum ingredient matches to keep a recipe
rerank_weight	0.6	Overlap fraction in combined_score (0.4 goes to semantic)
LOCAL_CONFIDENCE_THRESHOLD	0.35	Trigger Spoonacular if below
LLM_TRIGGER_THRESHOLD	0.25	Trigger LLM generation if below
Deduplication threshold	85%	Fuzzy title similarity cutoff
Calorie penalty divisor	500 kcal	Max acceptable calorie deviation
LLM temperature	0.8	Recipe generation creativity

Table 13 — Key tunable parameters

PlatePlanner

Models, Ingredient Breakdown & Recipe Generation

San Jose State University · CMPE 295B · April 2026